

Reviewer Pack — Bounded Arithmetic Proof Complexity and Resolution Length

Entry e5c763304a86 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-30 16:48:13 UTC

Bounded Arithmetic Proof Complexity and Resolution Length

Entry ID: e5c763304a86 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-30 16:48:13 UTC

1. Conjecture

Field A (mathematical branch): Bounded Arithmetic **Field B** (complexity object): Resolution Proof Length

Statement:

For a CNF formula F with n variables, the minimal resolution proof length is $\Theta(n^k)$ if and only if F is provable in the bounded arithmetic theory S^1_2 with k quantifier alternations.

Rationale (proposer’s reasoning):

Bounded arithmetic theories (S^1_2 , T^1_2) encode proof systems for polynomial-time reasoning. Resolution proof length captures propositional complexity, and their interplay reveals structural constraints on arithmetic strength. Quantifier alternations in S^1_2 correlate with proof depth, linking to resolution width.

Taxonomy category: BOUNDED_ARITHMETIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7412b79321de0d3a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: INCONCLUSIVE against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - bounded arithmetic resolution proof length S^1_2 quantifier alternations - proof complexity resolution length CNF formula S^1_2 bounded arithmetic - minimal resolution proof length quantifier alternations bounded arithmetic S^1_2

Top relevant hits considered: - [<http://arxiv.org/abs/0706.3841v1>] Arithmetic lattices and weak spectral geometry - [<http://arxiv.org/abs/1411.7087v6>] Consistency proof of a fragment of PV with substitution in bounded arithmetic - [<http://arxiv.org/abs/2411.15515v2>] Finding Bugs in Short Proofs: The Metamathematics of Resolution Lower Bounds - [<http://arxiv.org/abs/1804.02478v5>] Complexity of the CNF-satisfiability problem - [<http://arxiv.org/abs/1905.10867v2>] Regular resolution for CNFs with almost bounded one-sided treewidth - [<http://arxiv.org/abs/1606.05050v1>] Proof Complexity Lower Bounds from Algebraic Circuit Complexity - [<http://arxiv.org/abs/1405.2700v1>] Zero Excess and Minimal Length in Finite Coxeter Groups

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=241.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_3cnf(n):
        clauses = []
        for _ in range(2**n):
            clause = [random.randint(-1, 0)*i-1 for i in range(1, n+1)]
            if random.random() < 0.5:
                clause = [-x for x in clause]
            clauses.append(clause)
        return clauses

    def dpll(clauses, model):
        if not clauses:
            return True
        unit_clauses = [c for c in clauses if len(c) == 1]
        if unit_clauses:
            literal = unit_clauses[0][0]
            new_model = model.copy()
            new_model[literal] = True if literal > 0 else False
            if not dpll([c for c in clauses if literal not in c and -literal not in c], new_model):
                return False
        pure_literals = [l for l in range(1, n+1) if all(l not in c or -l not in c for c in clauses)]
        if pure_literals:
            literal = pure_literals[0]
            new_model = model.copy()
            new_model[literal] = True
            if not dpll([c for c in clauses if literal not in c and -literal not in c], new_model):
                return False
        for literal in range(1, n+1):
```

```

        if literal not in model and -literal not in model:
            new_model = model.copy()
            new_model[literal] = True
            if dpll([c for c in clauses if literal not in c and -literal not in c], new_model):
                return True
            new_model[literal] = False
            if dpll([c for c in clauses if literal not in c and -literal not in c], new_model):
                return True
    return False

def resolution(clauses):
    while True:
        new_clauses = []
        added = False
        for i in range(len(clauses)):
            for j in range(i+1, len(clauses)):
                common_literals = [l for l in clauses[i] if -l in clauses[j]]
                if common_literals:
                    new_clause = list(set([l for l in clauses[i] + clauses[j] if l not in common_literals]))
                    if new_clause not in new_clauses:
                        new_clauses.append(new_clause)
                        added = True
        if not added:
            break
        clauses += new_clauses
    return len(clauses)

n = random.randint(5, 40)
formula = generate_3cnf(n)
proof_length = resolution(formula)

# Simulate S12 provability by checking if the formula is a tautology or has a short Frege proof
def is_tautology(clauses):
    return dpll(clauses, {})

def frege_proof(clauses):
    stack = []
    for clause in clauses:
        stack.append(clause)
    while len(stack) > 1:
        new_clause = set()
        for c1 in stack:
            for c2 in stack:
                common_literals = [l for l in c1 if -l in c2]
                if common_literals:
                    new_clause = list(set([l for l in c1 + c2 if l not in common_literals and l not in new_clause]))
                    if new_clause not in stack:
                        stack.append(new_clause)
    return len(stack) == 1

s1_2_provable = is_tautology(formula) or frege_proof(formula)

conjecture_holds = proof_length == n**2 and s1_2_provable
counterexample = "" if conjecture_holds else "mapping_undefined"

return {

```

```

        "metric_name": "average_proof_length",
        "metric_value": proof_length,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    avg_proof_length = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={avg_proof_length:.2f} std=0.00 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results, preventing evaluation of support fraction or counterexamples. | next: Increase timeout duration and re-run test with stricter resource limits

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	108072
2	preregistration	ollama_remote	qwen3:8b	0	0	23864
3	novelty	ollama_remote	qwen3:8b	0	0	18987
4	novelty	ollama_remote	qwen3:8b	0	0	24415
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16714
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13148
7	judge	ollama_remote	qwen3:8b	0	0	90941

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 296141 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/e5c763304a86.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e5c763304a86.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/e5c763304a86.tar.gz` (if generated)